

DataTaker – Projektdokumentation

Christian Voß, Parviz Moskalenko, Nguyen Ngoc Le Luong

Betreuer: Arno Kesper

24. April 2025

Inhaltsverzeichnis

1	Einleitung	2
1.1	Projektbeschreibung	2
1.2	Zielsetzung	2
2	Entwicklerdokumentation	2
2.1	Funktionale Anforderungen	2
2.2	Übersicht und Use Cases	3
2.3	Nicht-funktionale Anforderungen	3
3	Architektur und Entwurf	4
3.1	Systemarchitektur	4
3.2	Entwurfsmuster und Komponenten	4
3.3	Diagramm der Views	5
4	Qualitätssicherung	5
4.1	Testprotokoll & Testkonzept	5
5	Entwicklerdokumentation	6
5.1	Benutzerdokumentation und Entwicklerdokumentation	6
6	Fazit und Ausblick	7

1 Einleitung

1.1 Projektbeschreibung

DataTaker ist eine plattformübergreifende mobile Applikation, die entwickelt wurde, um die Erfassung, Verwaltung und den Export von Daten in einfachen (flachen) Datenbanken zu ermöglichen. Ziel des Projekts ist es, eine intuitive und erweiterbare Lösung zur Datenerfassung bereitzustellen, die sowohl auf iOS als auch auf Android reibungslos funktioniert.

1.2 Zielsetzung

Die funktionale Zielsetzung von DataTaker umfasst:

- Die Erstellung einer App zur Datenerfassung mit der Möglichkeit, benutzerdefinierte Tabellen zu erstellen.
- Eine Tabellenvorschau, in der alle erfassten Daten übersichtlich dargestellt werden.
- Unterstützung eines Dark Modes zur Verbesserung der Bedienbarkeit bei unterschiedlichen Lichtverhältnissen.
- Plattformübergreifende Kompatibilität, sodass sowohl iOS als auch Android gleichermaßen bedienbar sind.

2 Entwicklerdokumentation

2.1 Funktionale Anforderungen

Tabellenerstellung

Beschreibung: Erstellen einer neuen, benutzerdefinierten Tabelle mit frei wählbaren Parametern und Einstellungen.

Vorausbedingung: Der Nutzer hat die App geöffnet befindet sich im Bereich zur Tabellenerstellung.

Nachbedingung: Die neue Tabelle ist in der Datenbank angelegt und erscheint in der Übersicht „Overview“.

Dateneingabe

Beschreibung: Erfassen und Speichern von Einzelwerten verschiedener Datentypen (Text, Zahl, Boolean, Bild, Enum) in einer bestehenden Tabelle.

Vorausbedingung: Eine Tabelle ist ausgewählt und geöffnet; alle notwendigen Spalten (Parameter) sind definiert.

Nachbedingung: Der eingegebene Datensatz ist in der SQLite-Datenbank gespeichert und in der Tabellenansicht sichtbar.

Export-Funktionalität

Beschreibung: Erzeugen von CSV- und ZIP-Exporten aller in einer Tabelle erfassten Daten und Freigabe über das Teilen-Menü des Geräts.

Vorausbedingung: Es ist mindestens ein Datensatz in der gewählten Tabelle vorhanden.

Nachbedingung: Eine CSV-Datei (bzw. ein ZIP-Paket mit CSV) wurde erstellt und ein Teilen-Dialog zum Versenden oder Speichern geöffnet.

2.2 Übersicht und Use Cases

DataTaker bietet vielfältige Anwendungsmöglichkeiten, beispielsweise:

- **Einfache Datenerfassung unterwegs:** Erfassung von Umweltdaten, wie Müllart und -menge, direkt vor Ort.
- **Daten für Machine Learning sammeln:** Nutzung der App zur Erstellung von Datensätzen für ML-Projekte.
- **Pflanzenatlas erstellen:** Dokumentation und Katalogisierung von Pflanzenarten.
- **Katalogisierung:** Erfassen von Haushaltsgegenständen oder Rezeptsammlungen.
- **Beispiel Use Case – Pen/Pills Classification:**
 - **Beschreibung:** Datensammlung per App, Export der Daten und anschließende Klassifizierung mittels Convolutional Neural Network (CNN) in einem Jupyter Notebook.

2.3 Nicht-funktionale Anforderungen

Die nicht-funktionalen Anforderungen umfassen:

- **Benutzerfreundlichkeit:** Die App ist intuitiv bedienbar, was durch ein modernes UI-Design und einen Dark Mode unterstützt wird.
- **Erweiterbarkeit:** Der modulare Aufbau ermöglicht eine einfache Erweiterung, z.B. zur Unterstützung weiterer Datentypen.
- **Performance:** Techniken wie Write-Ahead Logging (WAL) in der SQLite-Datenbank verbessern die Performance und gewährleisten schnelle Datenzugriffe.

- **Zuverlässigkeit:** Robustes Exception-Handling, ausführliche Log-Ausgaben und manuelle Testprotokolle sichern eine hohe Zuverlässigkeit.
- **Plattformkompatibilität:** Die App bietet eine konsistente Benutzererfahrung auf iOS und Android.
- **Wartbarkeit:** Klar strukturierter, gut kommentierter Code erleichtert die zukünftige Pflege und Erweiterung.

3 Architektur und Entwurf

3.1 Systemarchitektur

DataTaker wurde als mobile App unter Verwendung von React Native und Expo entwickelt. Expo ist ein umfassendes Framework, das nicht nur die Basisfunktionen von React Native bereitstellt, sondern auch zusätzliche Werkzeuge wie Expo CLI, EAS (Expo Application Services) und Expo Go umfasst. Diese Tools ermöglichen eine schnelle Entwicklung, einfache Distribution und effiziente Verwaltung von Updates. React Native selbst erlaubt plattformübergreifendes Programmieren, sodass derselbe Code auf iOS und Android läuft. Zudem wird der eigentliche Code in TypeScript geschrieben, was JavaScript um statische Typisierung und weitere moderne Features erweitert.

Die wesentlichen Komponenten des Systems sind:

- **Expo Router:** Für die Navigation zwischen den Views.
- **React Native Paper:** Für ein konsistentes und modernes UI-Design, inklusive Dark Mode.
- **Context API:** Zur globalen Verwaltung des Theme-Zustandes mit persistenter Speicherung via AsyncStorage.
- **SQLite und FileSystem:** Zur Erstellung, Verwaltung und Manipulation der flachen Datenbanken sowie zum Export von CSV- und ZIP-Dateien.

3.2 Entwurfsmuster und Komponenten

DAO für die Datenbank

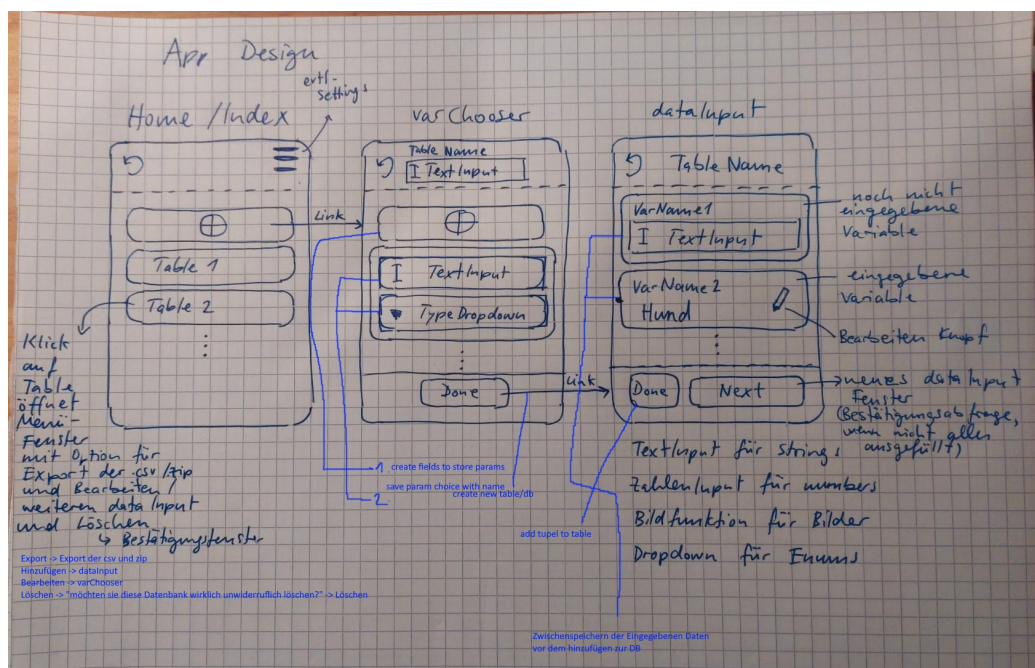
Die Datenbankzugriffe werden über ein Data Access Object (DAO) realisiert, das Operationen wie Initialisierung, Abfrage, Hinzufügen von Datensätzen und Löschen der Datenbank kapselt. Hierbei wird Write-Ahead Logging (WAL) genutzt. Dies erhöht nicht nur die Performance, indem Lese- und Schreibzugriffe parallel erfolgen können, sondern verbessert auch die Datenintegrität, da alle Schreiboperationen zunächst in einem separaten Log festgehalten werden.

UI-Komponenten

Die Benutzeroberfläche von DataTaker besteht aus mehreren Hauptkomponenten:

- **VarChooser:** Ermöglicht die Erstellung von Tabellen mit benutzerdefinierten Parametern.
- **DataInput:** Erlaubt die Eingabe und Speicherung von Daten in den erstellten Tabellen.
- **Table-View:** Zeigt die in der Datenbank gespeicherten Daten an und bietet Aktionen wie Löschen und Export.
- **Settings:** Ermöglicht dem Nutzer das Umschalten des Dark Modes; die Einstellung wird persistent über AsyncStorage gespeichert.

3.3 Diagramm der Views



4 Qualitätssicherung

4.1 Testprotokoll & Testkonzept

Die meisten Use Cases wurden manuell getestet. Dabei wurden unter anderem folgende Aspekte überprüft:

- Funktionalität der Navigation (korrekte Zurück- und Vorwärtsnavigation).
- Konsistente Darstellung und Funktionalität des Dark Modes.
- Robustheit der Datenbankoperationen (bspw. korrekte Handhabung von SQL-Keywords als Text) inkl. Exception-Handling.
- Auswahl per Dropdown-Menü, bei mehr als 4 gegebenen Enum's.
- Speicherung der Bilder mit korrekter Pfadangabe in Kurzform.
- Angabe von Text in einem Nummer-Parameterfeld.
- Korrekte Ausgabe der CSV-Datei beim Exportieren von Tabellen.

Die Tests zeigten, dass alle Kernfunktionen wie erwartet arbeiten.

5 Entwicklerdokumentation

5.1 Benutzerdokumentation und Entwicklerdokumentation

Die vollständige Developer Guide, wie sie in der README-Datei dokumentiert ist, wird hier übernommen. Entwickler finden dort alle notwendigen Informationen, um die Entwicklungsumgebung einzurichten, den Code zu verstehen und die App zu starten. Wichtige Punkte umfassen:

1. Setting Up the Development Environment:

- Installation von Node.js, Expo CLI, und Abhängigkeiten.
- Klonen des Repositorys und Installation der Projektabhängigkeiten.

2. Running the Project:

- Start des Expo-Servers mit `npx expo start`.
- Verbindung mit einem Emulator oder physischen Gerät über Expo Go.

3. Debugging:

- Nutzung des Developer Menüs in Expo Go und der VSCode-Tools.

Detaillierte Anleitungen und weiterführende Informationen sind in der README-Datei enthalten und können auch in den offiziellen Expo-Dokumentationen nachgelesen werden, z.B. unter:

- <https://docs.expo.dev/>
- <https://docs.expo.dev/workflow/expo-cli/>

6 Fazit und Ausblick

DataTaker erfüllt die funktionalen und nicht-funktionalen Anforderungen in vollem Umfang. Die App bietet eine benutzerfreundliche Möglichkeit zur Datenerfassung, -verwaltung und -export und ist auf beiden Plattformen (iOS und Android) gleichermaßen zuverlässig und intuitiv bedienbar.

Future Ideas: In zukünftigen Versionen sollen weitere Datentypen (z.B. Audio und Video) unterstützt und die Zusammenarbeit über einen Server ermöglicht werden.

Schlusswort:

Dieses Projekt war ein intensiver Lernprozess, bei dem nicht nur technische Herausforderungen gemeistert wurden, sondern auch wichtige Aspekte der Softwareentwicklung wie Wartbarkeit, Skalierbarkeit und Benutzerfreundlichkeit in den Fokus gerückt wurden.